Names, Bindings, Scopes, and Lifetimes

Lec4

30.12.2025

BAKD

DoCE

FoE

UoP

Chapter 5: Names, Bindings, and Scopes

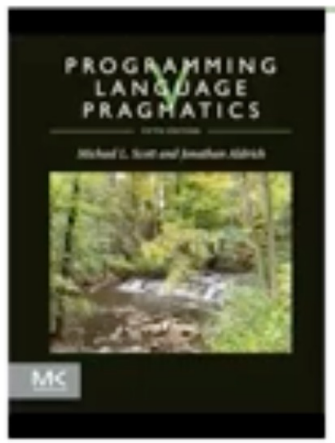
5.3: Variables
int varName = varValue

- Value: That which is stored within a variables memory. placed on the RHS of a declaration or assignment.
- Name: Names are the way that we identify variables. Immediately after type (varName in this case)
  - Aliases: One address can be referenced by two names, not good for reading code
- Address: Addresses are the computer memory cells that store information in them regarding the variable. This is not seen directly by programmers.
- Type: The type of information stored, this in part determines the amount of memory and how that information is stored. A long int will take up more space than a short or regular int. (The type in this case is int, it precedes the variable name. In some cases it is not needed)

5.6: Scope and Lifetime of Variables

- Scope is the "reach" that a variable has, where it may be accessed.
- Often restricted to the function range
- Scope is where the function can be reached,
  Lifetime is temporal and is the time that the memory for that variable exists and is allocated
- C++'s 'static' keyword extends lifetime beyond the life of the function, but doesn't alter scope.
- Both of these concepts become fuzzy with subprogram calls and in a number of other s
  from main.

Chapter 5: Names, Bindings, and Scopes



Programming Language Pragmatics, Fifth Edition

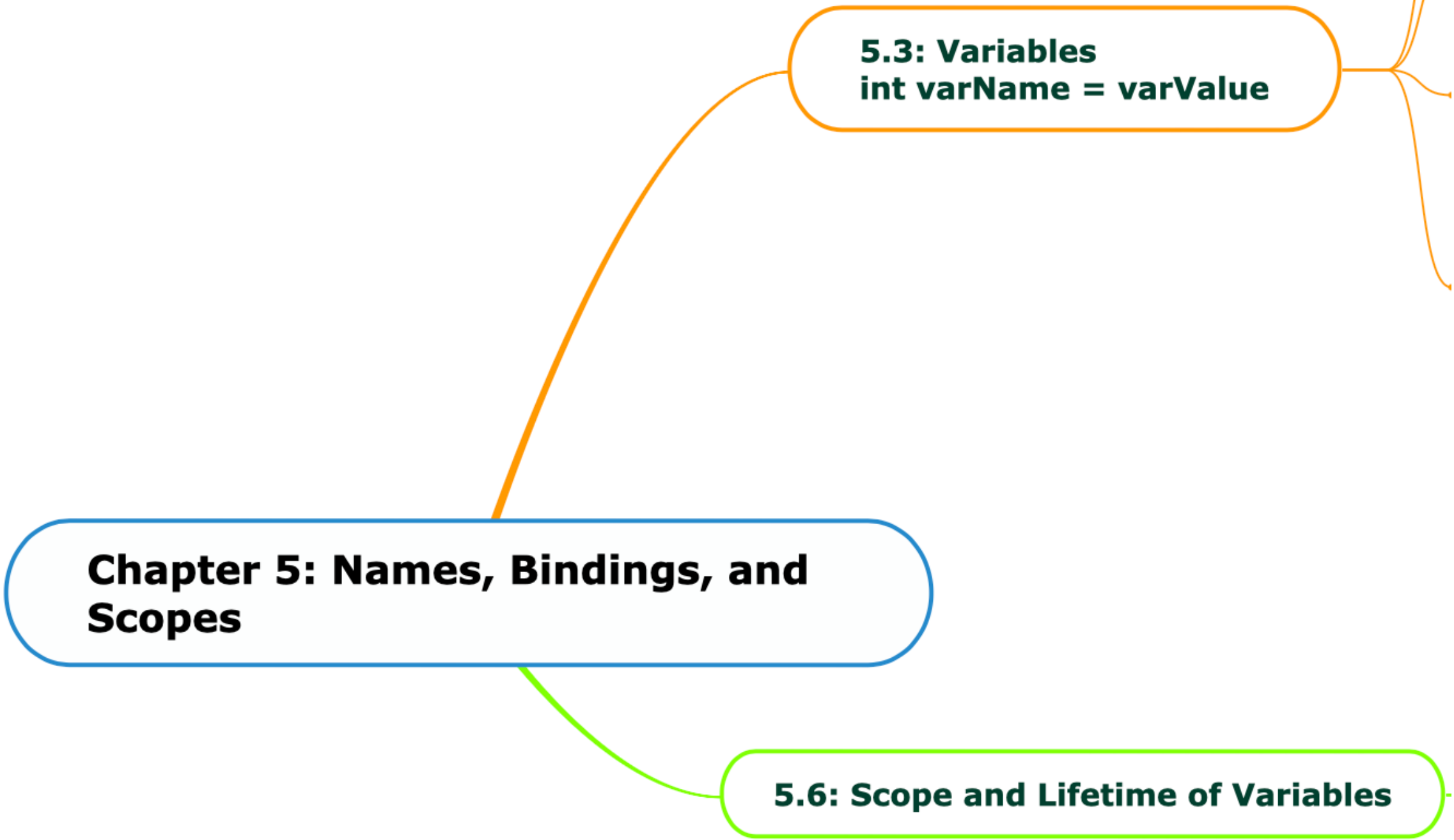
Michael L. Scott and Jonathan Aldrich

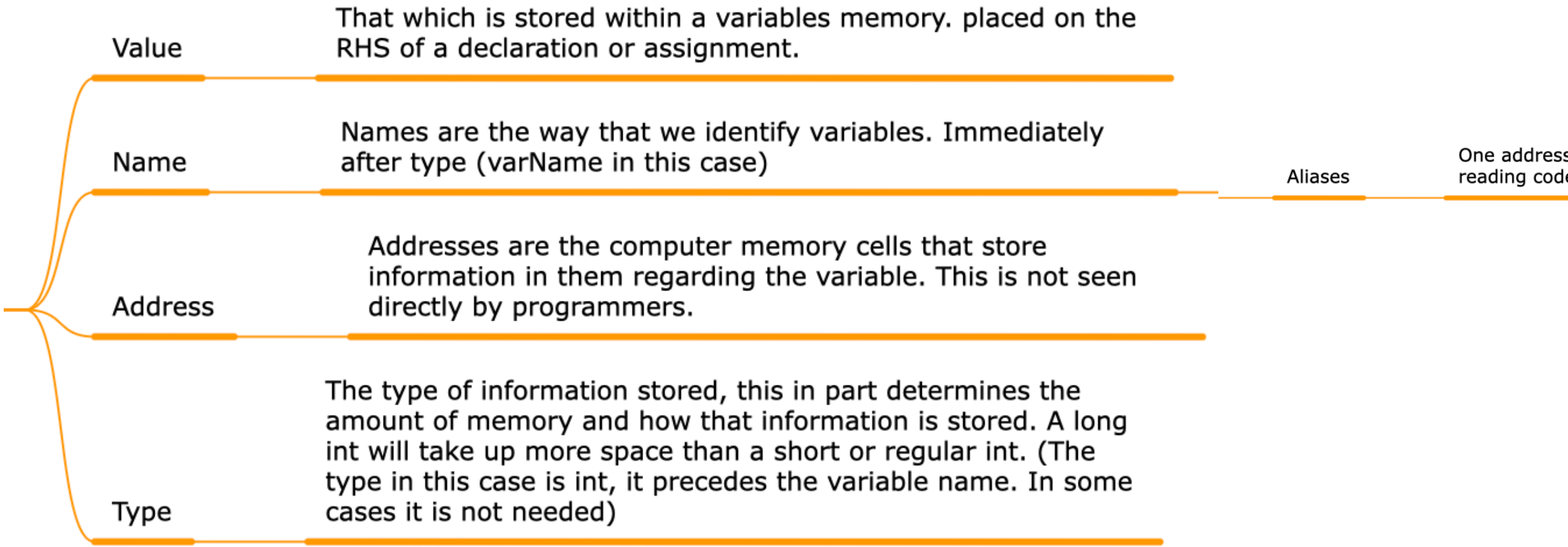
Names raise the level of abstraction of programs

- Can refer to variables, functions, etc. using symbolic identifiers instead of addresses

- Can use a name to refer to a more complicated structure
  - A subroutine's name abstracts the implementation code
  - A class's name abstracts the data representation

- Most program data is referred to by names
  - Data on the heap is an exception—it is referred to by pointers
    - But, those pointers are stored in variables that are named!


5.3: Variables
int varName = varValue
Chapter 5: Names, Bindings, and Scopes
5.6: Scope and Lifetime of Variables


Value
That which is stored within a variables memory. placed on the RHS of a declaration or assignment.
Name
Names are the way that we identify variables. Immediately after type (varName in this case)
Aliases
One address
reading code
Address
Addresses are the computer memory cells that store information in them regarding the variable. This is not seen directly by programmers.
Type
The type of information stored, this in part determines the amount of memory and how that information is stored. A long int will take up more space than a short or regular int. (The type in this case is int, it precedes the variable name. In some cases it is not needed)

Names, scopes, and binding

- Consider this example of a variable binding:

{

S_1

int x = *e*;

S_2

}

- x is a *name*
- **int** x = *e*; is a *binding*
  - associates x with a variable
  - assigns the result of evaluating *e* to the variable
- The *scope* of x is where the binding is active
  - typically the statements S_2 that follow the binding

Binding time

- The point at which a binding is created
  - A module import name is bound to an implementation at link time
  - A variable is bound to a value at run time
  - More generally, the point at which an implementation decision is made
  - Generally, decisions made before run-time are called *static*, decisions made at run time are *dynamic*

Decisions and their binding times

- Language design time
  - What language constructs and types are available
- Language implementation time
- Language implementation time
  - Representation and precision of primitive values, layout of the stack
    - How many bits are in a C int?
- Program writing time
  - Programmer's choice of algorithms, data structures, and names
- Compile time
  - Mapping of source code to machine code, layout of data structures

Decisions and their binding times (continued)

- Link time
  - Binding of a module's imports to the referenced modules
- Load time
  - Exact layout of code in memory
- Run time
  - Binding of variables to values

Binding time in compilers and interpreters

- Compilers make many decisions at compile time
  - This makes the run time more efficient, because these decisions are already made
- Interpreters delay many decisions until run time
  - Allows code to be more flexible, automatically supporting polymorphism
    - The type of data stored in each variable does not have to be determined in the source code, and can vary at run time

- What is the *scope* of a binding?

- Answer: the scope of a binding is where the binding can be used. For example, the scope of a local variable in most languages goes from where it is declared to the end of the current block.

Scope is the "reach" that a variable has, where it may be accessed.

Often restricted to the function range

Scope is where the function can be reached,
Lifetime is temporal and is the time that the memory for that variable exists and is allocated

C++'s 'static' keyword extends lifetime beyond the life of the function, but doesn't alter scope.

Both of these concepts become fuzzy with subprogram calls and in a number of other situations outside of function calls from main.

Scope is where the function can be reached, Lifetime is temporal and is the time that the memory for that variable exists and is allocated

Often restricted to the function range

Scope is the "reach" that a variable has, where it may be accessed.

Variables int varName = varValue

Type

The type of information stored, this in part determines the amount of memory and how that information is stored. A long int will take up more space than a short or regular int. (The type in this case is int, it precedes the variable name. In some cases it is not needed)

Address

Addresses are the computer memory cells that store information in them regarding the variable. This is not seen directly by programmers.

Name

Names are the way that we identify variables. Immediately after type (varName in this case)

Aliases

One address can be referenced by two names, not good for reading code

Value

That which is stored within a variables memory. placed on the RHS of a declaration or assignment.

Binding- A **binding** is the attachment of a name to an entity.

- Programs, are constructed from **entities**.
- Entities refer to, store, and manipulate values.
- Some entities are pure language structures, like declarations, expressions, and statements.
- Literals are entities that directly represent values. But other entities can be *named*.

Examples of names	Examples of entities
• `x` • `split` • `determine_best_move` • `determineBestMove` • `IllegalArgumentException` • `Integer` • `org.joda.time` • `/usr/bin/python` • `先生` • `+` • `<=>` • `|-*-|`	• Types • Variables • Labels • Blocks • Functions • Fields • Methods • Classes • Modules • Operators • Threads • Tasks • Processes • Parameters • Packages

Names allow us to **refer to** entities.

***Exercise**: Imagine trying to program without naming anything. Imagine trying to refer to yourself, your friends, your things, or anything, without the benefit of names. How might you compensate for a lack of names?*

- In most languages, the set of symbolic names are fixed, but in a few, like Scala, Swift, Haskell, and Standard ML, you can define your own.
- ***Exercise****: Find an example of defining your own symbolic names in one of these languages*.

Declarations vs. Definitions

A **declaration** brings a name into existence, but a **definition** creates an entity and the initial binding.

```
extern int z;                  // just a declaration
class C;                       // just a declaration
double square(double);         // just a declaration

void f() {                     // declaration plus definition together
  int x;                       // also a declaration plus definition
  int y = x + z /square(3); // so is this
}

double square(double x) {   // definition completing earlier declaration
  return x * x;
}
```

- The purpose of a definition-

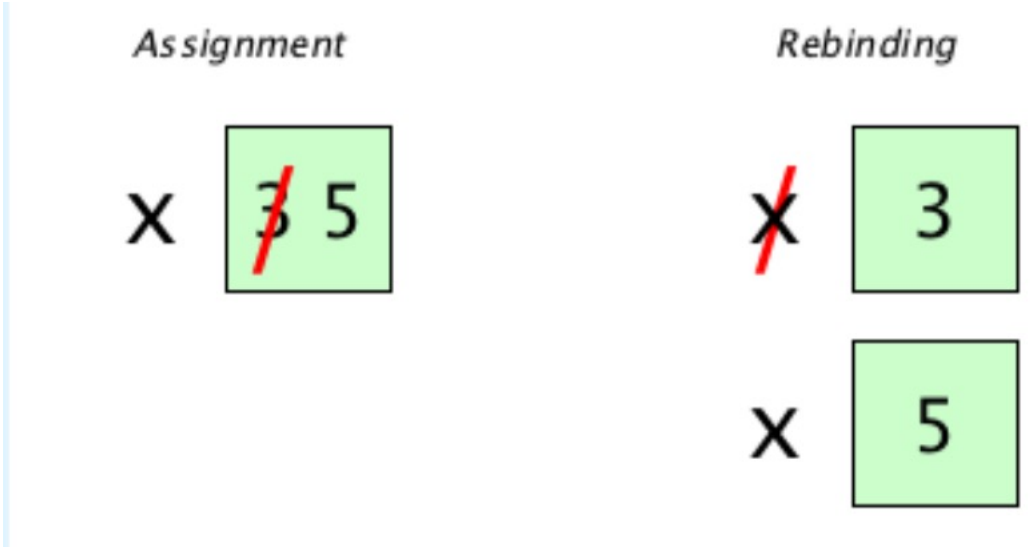
less declaration is to allow the name to used, even though the definition will be somewhere else (in a place you might not even know).

Assignments vs. Bindings

What do you think the meaning of the following is?

- x ← 3
- x ← 5

Is there one entity here? Or two?



- If one, we are changing the value of an entity, which we call **assignment**, and in which case we call the entity an **assignable**.

Here are two languages that do things differently. JavaScript does assignment:

```
let x = 3              // create entity, value is 3, bind x
const f = () => x      // f returns value of entity bound to x
x = 5                  // assignment: replace value of entity
f()                    // ==> 5
```

And Standard ML does rebinding:

```
val x = 3;             (* create entity, bind to x *)
val f = fn () => x;    (* f returns value of entity NOW bound to x *)
val x = 5;             (* COMPLETELY NEW BINDING *)
f();                   (* ==> 3, because value of original binding *)
```

***Exercise**: Do this little assignment vs. rebinding experiment in Python, Perl, Lua, Julia, Erlang, Go, Swift, Rust, etc., etc.*

Here are two languages that do things differently. JavaScript does assignment:

```
let x = 3              // create entity, value is 3, bind x
const f = () => x      // f returns value of entity bound to x
x = 5                  // assignment: replace value of entity
f()                    // ==> 5
```

And Standard ML does rebinding:

```
val x = 3;             (* create entity, bind to x *)
val f = fn () => x;    (* f returns value of entity NOW bound to x *)
val x = 5;             (* COMPLETELY NEW BINDING *)
f();                   (* ==> 3, because value of original binding *)
```

Initialization vs. Assignment

Initialization and assignment **are very different**.

- **Initialization** creates a variable where none existed before
- **Assignment** updates the value of an existing variable

One language that makes the distinction explicit in code is C++. Examples:

One language that makes the distinction explicit in code is C++. Examples:

```
// C++ Initializations:
int x = 10;
int y(15);
int z(a + 5 / 2);
int w(x);
Point p(5, 12);
Point q = p;

// C++ Assignments:
x = 12;
y = x / 6;
q = p;
q = midpoint(p1, p2);
```

Polymorphism and Aliasing

Instead of rebinding a name to a *new* entity, what if we allowed the *same name to refer to different entities at the same time*? Oooooh. And, and.... turning things around, what if we allowed *multiple names to be bound to the same entity at the same time*?

We have words for these things:

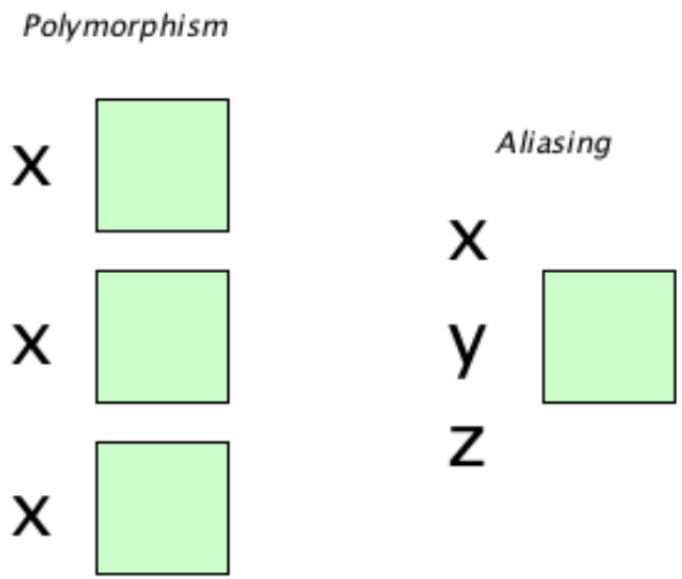
Polymorphism

Same name bound to multiple entities at the same time

Aliasing

Multiple names bound to the same entity at the same time

And pictures:



A precise way to say "at the same time" is "within the same referencing environment." Technically, a **referencing environment** *is the set of bindings in effect at a given point.*

Examples of Polymorphism

In a few languages, multiple functions can have the same name at the same time provided their signatures (parameter names, types, orders, etc.), or their receiver class, differs. Check out this C++ example:

```
bool f(string s) { return s.empty(); }
int f(int x, char c) { return x * c; }

int main() {
  cout << f("hello") << '\n';             // 0
  cout << f(75, 'w') << '\n';             // 8925
}
```

- Polymorphism is interesting because, when given a name, a program has to find the right entity for the name to use! If the right entity can be determined at compile time, like in the example above, we have **static polymorphism**. If it can only be determined at run time, we have **dynamic polymorphism**,

- If it can only be determined at run time, we have **dynamic polymorphism**, like in this Ruby example:

```ruby
class Animal; end
class Dog < Animal; def speak() = 'woof' end
class Rat < Animal; def speak() = 'squeak' end
class Duck < Animal; def speak() = 'quack' end

animals = (1..50).map{[Dog.new, Rat.new, Duck.new].sample}
animals.each {|a| puts a.speak}
```

Here's another example of dynamic polymorphism, this time in Julia. It may *look* like static polymorphism, but Julia checks those types at run time:

```julia
function f(s::String) reverse(s) end
function f(x::Int, y::Char) x * Int(y) end

print(f("Hello"))        # "olleH"
print(f(75, 'π'))        # 72000
```

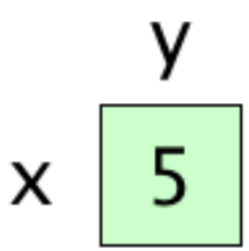
Evaluating that script in the console shows you Julia has "methods," a word commonly associated with dynamic lookup (we'll hear more about this when we cover OOP):

```
julia> function f(s::String) reverse(s) end
f (generic function with 1 method)

julia> function f(x::Int, y::Char) x * Int(y) end
f (generic function with 2 methods)

julia> f("Hello")
"olleH"

julia> f(75, 'π')
72000
```

A **pointer** is the name of a reference object that has a numeric address that you can manipulate with arithmetic operations.

```cpp
#include <iostream>

using namespace std;

int main() {
  int x = 5;
  int& y = x;  // x and y refer to the same entity

  x = 3;
  cout << x << ' ' << y << '\n';    // 3 3
  y = 8;
  cout << x << ' ' << y << '\n';    // 8 8
}
```

The picture for this one is easy:



Scope

Bindings have (generally spatial) scope and (temporal) extent. The **scope of a binding** is the *region in a program in which the binding is active*. We say scope is “generally” spatial because:

- Most languages employ **static scoping** (also called lexical scoping), meaning all scopes can be determined at compile time.
- Some languages (APL, Snobol, early LISP) employ **dynamic scoping**, meaning scopes depend on runtime execution.
- Perl lets you have both, but they suggested you never use dynamic scoping.

Regardless of whether scoping is static or dynamic, the deal is this: *when you leave a scope, the binding is deactivated*.

•

Static Scope

Static scoping is determined by the structure of the source code:

```
procedure P (X: Integer) is
   A, B: Integer;
   procedure Q (Y: Integer) is
      function R (X, Y: Integer) return Integer is
         procedure T is
            T: Integer;
         begin
            ...
         end T;
         W: Float;
      begin
         ...
      end R;

      procedure S (Y: Integer) is
      begin
         ...
      end S;
   begin
      ...
      declare X: Integer; begin ... end;
      ...
   end Q;
   M: Integer;
begin
   ...
end P;
```

- The general rule is that bindings are determined by finding the "innermost" declaration, searching outward through the *enclosing* scopes if needed.

- Where *exactly* the scope of a particular binding begins and ends
- Whether bindings that have been *hidden* in inner scopes are still accessible
- What actually counts as a *declaration* versus just a use? Can declarations be implicit?
- The effect of redeclarations within a scope

Scope Range

Where exactly a static scope begins and ends has *at least four* reasonable possibilities:

- Scope is whole block (like JavaScript let or const), or the whole function (like JavaScript var)
- Scope starts at beginning of declaration (like C)
- Scope starts only after the whole declaration is finished (like Lua)
- Scope starts after declaration is finished, but using occurrences of name are prohibited during declaration (like Ada)

Scope Holes

- In static scoping, “inner” declarations hide, or **shadow** outer declarations of the same identifier, causing a **hole**in the scope of the outer identifier’s binding.

Indicating Declarations

- Generally, languages indicate declarations with a keyword, such as var, val, let, const, my, our, or def. Or sometimes we see a type name, as in:

```c
int x = 5;
```

or

```
X: Integer := 3;
```

When we use these **explicit declarations** in a local scope, things are pretty easy to figure out, as in this JavaScript example:

```js
let x = 1, y = 2;
function f() {
  let x = 3;           // local declaration
  console.log(x);      // uses local x
  console.log(y);      // uses global y, it's automatically visible
}
f();                   // writes 3 and 2
```

Dynamic Scope

- With dynamic scoping, the current binding for a name is the one **most recently encountered during execution**. That is, scope becomes temporal rather than spatial.

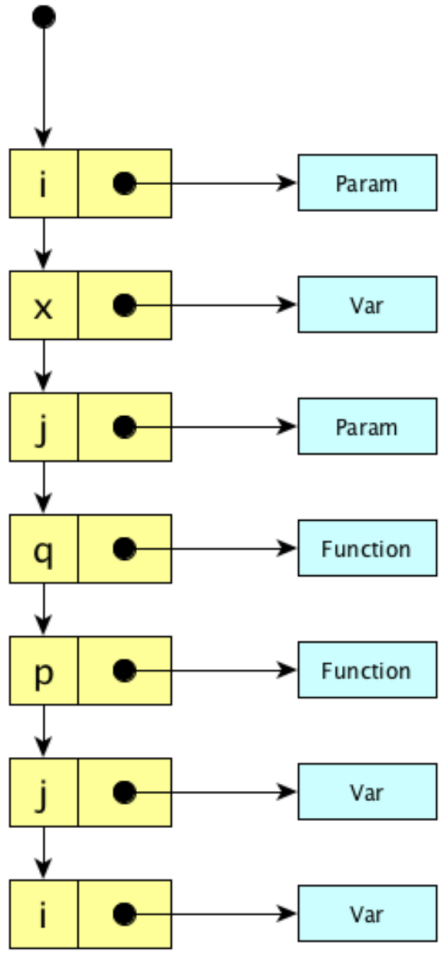
```perl
# Perl
our $x = 1;

sub f {
    print "$x\n";
}

sub g {
    local $x = 2;     # `local` makes dynamic scope (use `my` for static)
    f();
}

g();
print "$x\n";
```

Dynamic scopes tend to require *a lot* of run time overhead. The runtime system has to keep track of names in a structure that can be searched at run time to find the desired entity. There are two kinds of structures typically used to manage such scoping:

- Association lists
- Central reference tables

The **association list**, or A-list, works like this:

- When entering a scope: push its bindings on the list.
- When leaving a scope: pop its bindings off the list.
- When looking up a name: traverse the list.

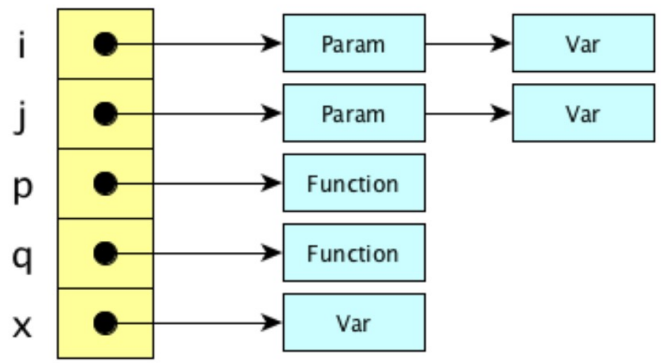


Generally speaking, scope entry and exit is fast, lookup is slow.

***Exercise**: Really? What exactly is the entry/exit time proportional to? What exactly is the lookup time proportional to?*

The **central reference table** works like this:

- When entering a scope: push an **entity descriptor** onto each chain corresponding to each name defined in the scope.
- When leaving a scope: pop the relevant entity descriptors.
- When looking up a name: do a constant-time hashtable lookup. (It's guaranteed to be $\Theta(1)$ because the correct entry is first on the list!)



Generally speaking, scope entry and exit is slow, lookup is blazingly fast (assuming your hash function is cheap).

***Exercise**: Really? What exactly is the entry/exit time proportional to?*

Lifetimes

The **lifetime** of an entity (also known as its **extent**) is just what it sounds like: the period (of time) from its creation (allocation) to its destruction (deallocation). With regard to lifetime, we can imagine different kinds of entities:

1. Global entities that live forever ("forever" means from the time of its creation to the end of the entire program)
2. Local entities created automatically when their enclosing function or block is called and destroyed when their containing function or block exits
3. Entities explicitly created by the programmer that must be explicitly destroyed by the programmer.
4. Entities explicitly created by the programmer and destroyed automatically when the runtime system determines they can no longer be referenced

In the world of programming language implementation, we often see storage partitioned into regions for different types of entities based on their lifetimes:

Region	Entities are...	Examples	Notes
Static	Allocated at fixed location in memory when program starts and live forever	• Global variables • String and floating point constants	• May or may not be present in binary image of program
Stack	Allocated and deallocated automatically at subroutine calls and returns	• Locals • Parameters • Temporaries	• Unless part of closure, can be stack-managed because of the way subroutine calls and returns behave
Heap	Allocated and deallocated dynamically	• Things made with `new` or `malloc` • Object and arrays made from literals or other expressions	• Allocation can be implicit or explicit • Deallocation can be implicit (e.g., garbage collection) or explicit • Watch out for memory leaks and dangling references

Video lesson

- https://youtu.be/q1pzUkEg87g?si=J8uqKkR4XLF1g6bP